RTL-to-GDS on GF180MCU

LibreLane + Docker in practice

EDA-Agents Project

April 22, 2026

Open-Source IC Design

RTL-to-GDS on GF180MCU
LibreLane + Docker in practice
EDA-Agents Project
April 22, 2026
Open-Source IC Design

What you will learn

- What RTL-to-GDS means end-to-end, and where LibreLane sits.
- How to get the full toolchain in **one Docker command**.
- How to configure LibreLane for a GF180MCU full-chip design.
- How to run the flow and read DRC / LVS / timing signoff.

What you will learn
- What RTL-to-GDS means end-to-end, and where LibreLane sits.
- How to get the full toolchain in **one Docker command**.
- How to configure LibreLane for a GF180MCU full-chip design.
- How to run the flow and read DRC / LVS / timing signoff.

Speaker build: notes on right-hand screen.

Agenda

The RTL-to-GDS pipeline

The stack, the container, the PDK

LibreLane config – the heart of the flow

Running the flow

Flow stages up close

Results, pitfalls, and next steps

Agenda
The RTL-to-GDS pipeline
The stack, the container, the PDK
LibreLane config – the heart of the flow
Running the flow
Flow stages up close
Results, pitfalls, and next steps

The RTL-to-GDS pipeline

The RTL-to-GDS pipeline

RTL and GDS – two ends of the same pipeline

RTL

A text description of a circuit in Verilog / SystemVerilog.
Says *what* the circuit does.

GDS / GDSII

A binary layout file – every polygon on every layer.
What the foundry turns into masks.

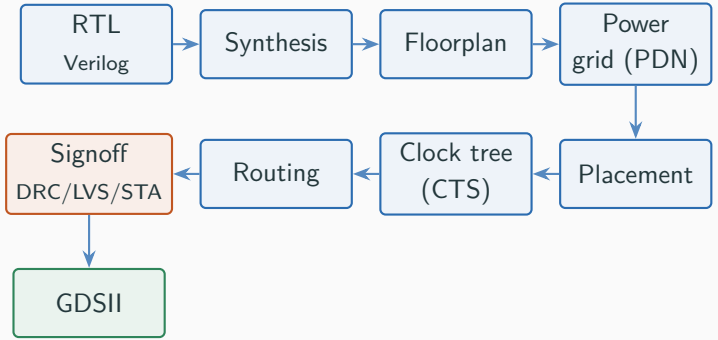
RTL-to-GDS = automating the path from one to the other.

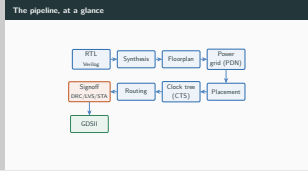
RTL and GDS – two ends of the same pipeline
RTL
A text description of a circuit in Verilog / SystemVerilog.
Says what the circuit does.
GDS / GDSII
A binary layout file – every polygon on every layer.
What the foundry turns into masks.
RTL-to-GDS = automating the path from one to the other.

Audience calibration. Most first-timers understand code (RTL) and know chips are "physical" (GDS) but have never seen the in-between steps. Emphasise the distinction: *behaviour* vs *geometry*. One wrong polygon and the mask is unusable; one wrong environment variable and the tool cannot even start. The whole deck earns its keep by showing this translation is now reproducible in one command.

Software analogy (spoken, not on the slide): RTL is like C source, GDS is like an ELF binary, and LibreLane is a compiler chain of ∼80 passes. Unlike software, each pass can veto manufacturability.

The pipeline, at a glance





Reading guide for the diagram.

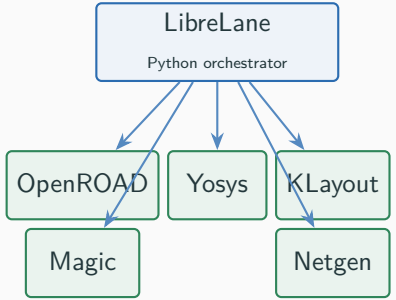
Synthesis (Yosys) turns RTL into a gate-level netlist of standard-cell instances. Floorplan decides die size and where macros / I/O sit. PDN draws VDD/VSS straps and rings. Placement legalises every cell to a standard-cell row. CTS balances the clock tree. Routing draws every signal wire. Signoff verifies manufacturability (DRC), schematic fidelity (LVS), and timing (STA).

Emphasise: each stage writes a milestone artifact, and any stage can halt the flow. LibreLane is the thing that stitches all 80-ish steps together – the rest of this deck is about driving *it*, not the individual tools.

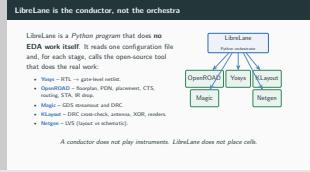
LibreLane is the conductor, not the orchestra

LibreLane is a *Python program* that does **no EDA work itself**. It reads one configuration file and, for each stage, calls the open-source tool that does the real work:

- **Yosys** – RTL → gate-level netlist.
- **OpenROAD** – floorplan, PDN, placement, CTS, routing, STA, IR drop.
- **Magic** – GDS streamout and DRC.
- **KLayout** – DRC cross-check, antenna, XOR, renders.
- **Netgen** – LVS (layout vs schematic).



A conductor does not play instruments. LibreLane does not place cells.



This slide is the thesis of the tutorial: *LibreLane is an orchestrator*, not an EDA tool. Students routinely conflate the two because "`librelane config.yaml`" is the only command they type. Make the distinction loud and early.

Say it out loud: "Yosys synthesises, OpenROAD places and routes, Magic writes GDS, KLayout cross-checks, Netgen does LVS. LibreLane decides *which one runs when* and *with what arguments*."

Two consequences: (1) errors almost always come from the underlying tool; the fix is in that tool's log, not in LibreLane's configuration. (2) If you already know any of those tools, you can override a step and keep the rest of the flow. LibreLane stays out of your way.

Pitch: this also explains why the open-source flow is bigger than it looks – five independent EDA projects, each with its own history, working together through one YAML file.

What you get at the end

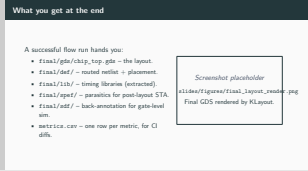
A successful flow run hands you:

- `final/gds/chip_top.gds` – the layout.
- `final/def/` – routed netlist + placement.
- `final/lib/` – timing libraries (extracted).
- `final/spef/` – parasitics for post-layout STA.
- `final/sdf/` – back-annotation for gate-level sim.
- `metrics.csv` – one row per metric, for CI diffs.

Screenshot placeholder

`slides/figures/final_layout_render.png`

Final GDS rendered by KLayout.



This frame sets the "you will have something tangible" expectation. The final layout render is what people put on posters. Everything else (DEF, SPEF, SDF) matters only to the people doing follow-up work: gate-level sim, analog post-layout, shuttle submission.

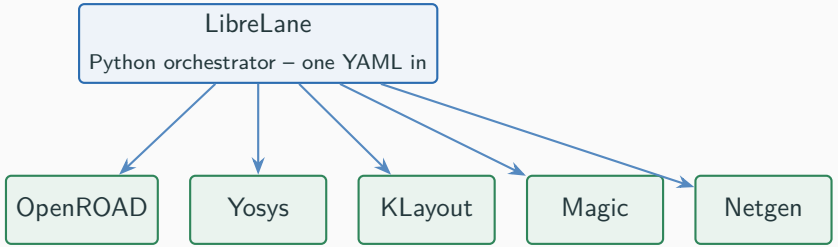
Name-drop `metrics.csv` – that is the single file you grep to know whether the chip is green.

Mention: commit a trimmed `metrics.csv` to git as a regression baseline; you will thank yourself when the image is rebuilt weeks later.

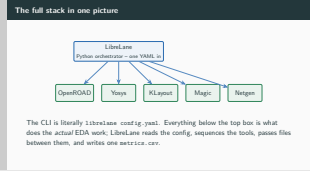
The stack, the container, the PDK

The stack, the container, the PDK

The full stack in one picture



The CLI is literally `librelane config.yaml`. Everything below the top box is what does the *actual* EDA work; LibreLane reads the config, sequences the tools, passes files between them, and writes one `metrics.csv`.

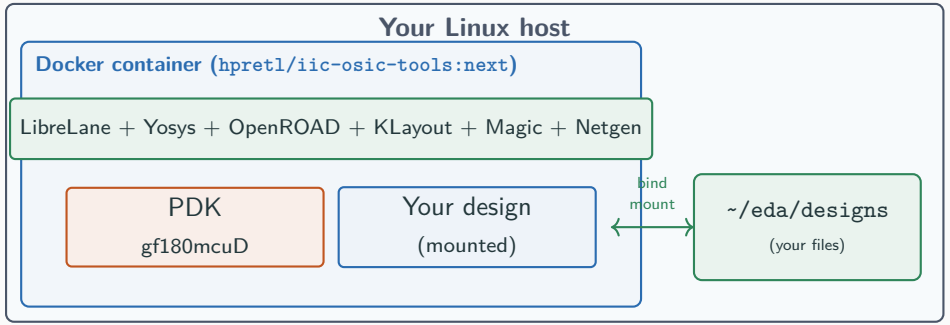


This restates the thesis from section 1 before we zoom into the container and PDK. The message: there are *six* independent open-source projects on this slide, not one. LibreLane, Yosys, OpenROAD, Magic, KLayout, Netgen are all maintained separately, each with its own release cadence and documentation.

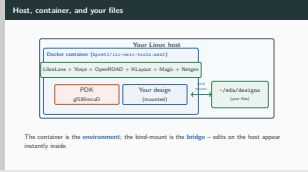
Name LibreLane as the evolution of OpenLane 2 (same lineage, Python rewrite). Audiences will almost never type `yosys ...` or `openroad ...` directly – but when a step fails, the log you read is the underlying tool's log, not a LibreLane log.

The upcoming two slides (container + PDK) are about giving these six tools a common place to live and a common set of foundry files to read.

Host, container, and your files



The container is the **environment**; the bind-mount is the **bridge** – edits on the host appear instantly inside.



Mental model is the whole point here. People comfortable with VMs get it in 30 seconds; people who have never used containers need to hear: the container is disposable, your host folder is not. Treat `~/eda/designs` as the canonical workspace. Everything written inside `/foss/designs` survives `docker rm`. Everything outside that path inside the container is gone.

The bind-mount is the only reason we can run a heavy-weight image and still keep our editor, git, and artifacts on the host where we already have backups.

Container bootstrap – four commands

```
# 1. One-time image pull (~15 GB)
docker pull hpretl/iic-osic-tools:next

# 2. Start a long-running, headless container
docker run -d --name gf180 \
    -v ~/eda/designs:/foss/designs:rw \
    --user $(id -u):$(id -g) \
    hpretl/iic-osic-tools:next --skip sleep infinity

# 3. (Optional) interactive shell for debugging
docker exec -it gf180 bash

# 4. Non-interactive use (what the rest of the deck does)
docker exec gf180 bash -lc '<command>'
```


Container bootstrap – four commands

Four commands is all a first-time user needs. The flags matter: `-v` binds the workspace, `-user` keeps artifacts on the host owned by the right UID, `-skip sleep infinity` is the image's built-in "do nothing, stay alive" entrypoint so we can `docker exec` into it later.

The `bash -lc` pattern is the non-interactive workhorse we use for every subsequent command; it yields a login shell so the container's `$PATH`, `$PDK_ROOT`, etc. are set up. Use single quotes on the host side so the entire payload is handed to the container shell untouched.

Cleanup (not on slide): `docker stop gf180 && docker rm gf180` removes the container but leaves your host files untouched.

GF180MCU: two PDK gotchas to know upfront

- The image ships `gf180mcuD` (5 V MCU flavour) via **ciel**. Good to go.
- The full-chip template needs two extra I/O cells (`gf180mcu_ws_io__dvdd`, `__dvss`) that live in the **wafer-space fork**:

```
git clone --depth 1 --branch 1.8.0 \
  https://github.com/wafer-space/gf180mcu.git gf180mcu
```

- The container defaults to IHP SG13G2. Every new shell must re-activate GF180:

```
source sak-pdk-script.sh gf180mcuD gf180mcu_fd_sc_mcu7t5v0
```


GF180MCU: two PDK gotchas to know upfront
The image ships gf180mcuD (5 V MCU flavour) via ciel. Good to go.
The full-chip template needs two extra I/O cells (gf180mcu_ws_io__dvdd, __dvss) that live in the wafer-space fork:
The container defaults to IHP SG13G2. Every new shell must re-activate GF180:

This is the single slide that saves every first-timer 2 hours of debugging. Without the wafer-space fork, you get an obscure "unknown cell" error during synthesis. Without the `sak-pdk-script` activation, LibreLane cheerfully looks for `sg13g2_stdcell` files inside the GF180 directory and stops on a Tcl glob error.

Both issues show up in Section 6 "Pitfalls" with their exact error messages – mention that now so people recognise the symptoms later. Tag 1.8.0 of the wafer-space fork is the version we audited; bump only with intent.

A **PDK** itself is just foundry files (stdcell libs, LEF, Liberty, SPICE models, DRC decks). We do not need to explain it in depth for a first-timer; the container already has the bits.

LibreLane config – the heart of the flow

LibreLane config – the heart of the flow

The project template: files you actually touch

The YAML below is LibreLane's input. LibreLane parses it and hands the relevant pieces to Yosys, OpenROAD, Magic, KLayout, and Netgen.

Base: `github.com/wafer-space/gf180mcu-project-template`.

Five files matter:

File	Purpose
`src/chip_top.sv`	Top module: padring + core.
`src/chip_core.sv`	Your logic. *Replace this for a custom chip.*
`librelane/config.yaml`	Design-wide settings (RTL, clock, PDN, macros).
`librelane/slots/slot_1x1.yaml`	Floorplan (die, core, pad order).
`librelane/pdn_cfg.tcl`	PDN script (trust it for now).

Plus a `Makefile` that wraps `librelane` in `make librelane`.

The project template: files you actually touch
The YAML below is LibreLane's input. LibreLane parses it and hands the relevant pieces to Yosys, OpenROAD, Magic, KLayout, and Netgen.
Base: github.com/wafer-space/gf180mcu-project-template.
Five files matter:
File | Purpose
src/chip_top.sv | Top module: padring + core.
src/chip_core.sv | Your logic. Replace this for a custom chip.
librelane/config.yaml | Design-wide settings (RTL, clock, PDN, macros).
librelane/slots/slot_1x1.yaml | Floorplan (die, core, pad order).
librelane/pdn_cfg.tcl | PDN script (trust it for now).
Plus a Makefile that wraps librelane in make librelane

The template is a *working* full-chip design – a tiny 42-bit counter with two SRAM512x8 macros sitting inside a padring on a 3.93×5.12 mm die. Clone it into the mounted workspace with `git clone -depth 1 https://github.com/wafer-space/gf180mcu-project-template.git template`.

The five files above are the surface area of the flow. Everything else (flake.nix, scripts/, ip/, cocotb/) is either nix-only (we ignore) or auxiliary (render helper, cocotb sim). Anchor the rest of this section around those five paths.

LibreLane v3 YAML – two levels, one command

- `config.yaml` – **design-level** (clock, RTL files, PDN, macros). Rarely changes.
- `slots/slot_*.yaml` – **floorplan-level** (die size, pad order). Changes per shuttle slot.

LibreLane merges both at the command line:

```
librelane librelane/slots/slot_1x1.yaml librelane/config.yaml \
        --pdk gf180mcuD --pdk-root <path> --manual-pdk
```


LibreLane v3 YAML – two levels, one command
config.yaml – design-level (clock, RTL files, PDN, macros). Rarely changes.
slots/slot_*.yaml – floorplan-level (die size, pad order). Changes per shuttle slot.
LibreLane merges both at the command line:
librelane librelane/slots/slot_1x1.yaml librelane/config.yaml \
--pdk gf180mcuD --pdk-root <path> --manual-pdk

Why two files? Because one template usually targets many shuttle slots (different die sizes). Keep everything die-shape-specific in `slot_*.yaml` and the rest in `config.yaml`. LibreLane reads the left-most file first and lets the right-most override, so list slots first, design second. `-manual-pdk` tells LibreLane not to activate ciel automatically; we did that ourselves with `sak-pdk-script.sh`. Every `meta.version: 3` flag on the slide reminds readers this is v3 syntax – v2 examples on the internet are outdated.

config.yaml – the header

```yaml
meta:
  version: 3
  flow: Chip

DESIGN_NAME: chip_top
VERILOG_FILES:
  - dir::../src/chip_top.sv
  - dir::../src/chip_core.sv

VDD_NETS: [VDD]
GND_NETS: [VSS]

CLOCK_PORT: clk_PAD
CLOCK_NET:  clk_pad/Y
CLOCK_PERIOD: 40           # ns -> 25 MHz
```


config.yaml – the header

`meta.flow: Chip` selects the full-chip padring flow (vs the smaller `Classic` flow used for bare-die blocks). `dir::` resolves paths relative to the YAML file – portable across machines. `CLOCK_PORT` is the external pad (`clk_PAD`); `CLOCK_NET` is the internal net after the pad cell's buffer (`clk_pad/Y`). If you only define `CLOCK_PORT`, CTS skews everything from the pad input and you get sad timing at the core.

Clock period drives synthesis effort and routing congestion. The reference run uses 40 ns (25 MHz) – comfortable slack for a 180 nm process. Tighten cautiously.

config.yaml – the power grid

```yaml
PDN_VWIDTH: 5
PDN_HWIDTH: 5
PDN_VSPACING: 1
PDN_HSPACING: 1
PDN_VPITCH: 75
PDN_HPITCH: 75

PDN_CORE_RING: true
PDN_CORE_RING_VWIDTH: 25
PDN_CORE_RING_HWIDTH: 25
PDN_CORE_RING_CONNECT_TO_PADS: true

PDN_CORE_VERTICAL_LAYER:   Metal2
PDN_CORE_HORIZONTAL_LAYER: Metal3

PDN_CFG: dir::pdn_cfg.tcl
```

Power is where designs die silently. Definitions:

- **Strap**: a wide VDD/VSS wire crossing the core.
- **Pitch**: the distance between neighbouring straps; larger = less metal, higher IR drop.
- **Core ring**: the loop of wide power wires around the standard-cell area; it delivers current from the pads to the straps.

The defaults here pass IR-drop for the reference design. If you scale up the core area, either shrink pitch or widen straps. LibreLane's PDNSim will flag violations in signoff.

`pdn_cfg.tcl` is OpenROAD-specific Tcl (voltage domains, per-macro PDN grids). Trust the template's version for now; revisit only when you change macro count or placement.

config.yaml – declaring macros

```yaml
MACROS:
  gf180mcu_fd_ip_sram__sram512x8m8wm1:
    gds:  [pdk_dir::libs.ref/gf180mcu_fd_ip_sram/gds/..gds]
    lef:  [pdk_dir::libs.ref/gf180mcu_fd_ip_sram/lef/..lef]
    vh:   [pdk_dir::libs.ref/.../sram__blackbox.v]
    lib:
      "*_tt_025C_5v00": [pdk_dir::.../sram__tt_025C_5v00.lib]
      "*_ss_125C_4v50": [pdk_dir::.../sram__ss_125C_4v50.lib]
      "*_ff_n40C_5v50": [pdk_dir::.../sram__ff_n40C_5v50.lib]
    instances:
      i_chip_core.sram_0: {location: [490, 500], orientation: N}
      i_chip_core.sram_1: {location: [970, 500], orientation: E}
```


config.yaml – declaring macros

A **macro** is a pre-characterised hard block (SRAM, analog IP, PLL). LibreLane needs four views: GDS (layout), LEF (abstract placement/routing shape), Verilog blackbox (for synthesis to stub it out), and one Liberty timing file per PVT corner.

`instances:` pins each macro instance at an absolute coordinate (microns) with an orientation (`N`, `S`, `E`, `W`, flipped variants `FN`, `FE` etc.). Manual placement is deliberate – automatic macro placement often collides with the padring.

The `pdk_dir::` prefix resolves against `PDK_ROOT`, so these paths stay portable.

Slots: a wafer-space convention, not a LibreLane one

A **slot** is a pre-reserved rectangle on wafer-space's MPW shuttle. Three are standard: `1x1` (∼3.9×5.1 mm), `1x0p5`, `0p5x0p5`. Each `slot_*.yaml` sets `DIE_AREA`, `CORE_AREA`, and the pad lists; everything else stays in `config.yaml`.

Not taping out with wafer-space? Three paths:

1. **Other shuttle** (IHP MPW, TinyTapeout, . . .) – use that programme's template; its own die sizes replace slots.
2. **Custom tape-out with padring** – drop `slots/`. Move `FP_SIZING`, `DIE_AREA`, `CORE_AREA` and the `PAD_*` lists into `config.yaml`. Pick any die size that fits your logic.
3. **Bare macro (no padring)** – switch to `meta.flow: Classic`. No pads, no slots, no sealring, no wafer-space PDK fork. That is what the counter notebook does.

Slots: a wafer-space convention, not a LibreLane one

A slot is a pre-reserved rectangle on wafer-space's MPW shuttle. Three are standard: 1x1 (∼3.9×5.1 mm), 1x0p5, 0p5x0p5. Each slot_*.yaml sets DIE_AREA, CORE_AREA, and the pad lists; everything else stays in config.yaml.

Not taping out with wafer-space? Three paths:

1. Other shuttle (IHP MPW, TinyTapeout, . . .) – use that programme's template; its own die sizes replace slots.
2. Custom tape-out with padring – drop slots/. Move FP_SIZING, DIE_AREA, CORE_AREA and the PAD_* lists into config.yaml. Pick any die size that fits your logic.
3. Bare macro (no padring) – switch to meta.flow: Classic. No pads, no slots, no sealring, no wafer-space PDK fork. That is what the counter notebook does.

Key message: *slot* is not LibreLane vocabulary and it is not a GF180 requirement. It exists because wafer-space's MPW shuttle organises projects as fixed-size rectangles on a shared wafer, and each project reclaims one such rectangle.

The template's split into `slots/slot_NxM.yaml` + `config.yaml` is a *convenience* for targeting several die sizes with the same design. For any other tape-out path you can flatten the two files into one.

Implication for this deck: every metric we quote (10 % core utilisation, 58 pads, 251k total instances) is a consequence of choosing `slot_1x1`. A smaller slot would mean fewer pads and tighter core; a custom die would mean whatever you set.

Option 3 (`Classic` flow) is the fastest RTL-to-GDS path – 1-2 min for the counter vs 35-45 min for the full chip. Reach for it whenever you do not need a padring; reach for the full-chip template only when you are actually going to bond the die.

slot_1x1.yaml – the floorplan

```yaml
FP_SIZING:  absolute
DIE_AREA:   [0, 0, 3932, 5122]        # um
CORE_AREA:  [442, 442, 3490, 4680]    # um

PAD_SOUTH: [clk_pad, rst_n_pad,
            "bidir\\[0\\].pad", ..., "dvss_pads\\[0\\].pad", ...]
PAD_EAST:  [...]
PAD_NORTH: [...]
PAD_WEST:  [...]
```

- `DIE_AREA` = full die outline (includes the sealring).
- `CORE_AREA` = rectangle for standard cells, inside the padring.
- Pad lists read *clockwise* starting from the south-west corner; backslashes escape the Verilog brackets (regex).

The padring is the single hardest thing to get right on a chip flow. The pad lists describe the physical order on each side; LibreLane's `OpenROAD.Padring` step walks them in sequence. Miss one pad and the padring won't close.

Coordinates are in microns, origin at the die's bottom-left corner. The gap between `DIE_AREA` and `CORE_AREA` is where the padring sits – roughly 440 um here, typical for 180 nm.

Change the slot, change the die – everything else stays the same. That separation is why LibreLane enforces two YAML files.

Running the flow

Running the flow

The golden path – copy, paste, done

```
docker exec gf180 bash -lc '
  cd /foss/designs/template

  # One-time: clone the wafer-space PDK fork
  make clone-pdk

  # Every shell: activate GF180MCU
  source sak-pdk-script.sh gf180mcuD gf180mcu_fd_sc_mcu7t5v0

  # Run the full RTL-to-GDS flow on slot 1x1
  make librelane \
       SLOT=1x1 \
       PDK=gf180mcuD \
       PDK_ROOT=/foss/designs/template/gf180mcu
'
```

Expected runtime: ∼35–45 min on a modern laptop.

The golden path – copy, paste, done
Expected runtime: ∼35–45 min on a modern laptop.

This is the entire flow. Three conceptual actions: (1) fetch the PDK fork, (2) activate the env, (3) invoke LibreLane through the template's Makefile. The `PDK_ROOT` on the command line overrides whatever the Makefile default is – deliberately redundant with `sak-pdk-script` to catch the trap we discuss in Section 6.

Wall time breakdown on the audit run: Yosys 6 s, floorplan 15 s, PDN + placement + CTS + routing ∼3 min, Magic DRC 32 min, KLayout antenna 7 min, LVS + STA + IR 1 min. If anyone asks "why so slow": Magic DRC on 84k standard cells with padring. The `make librelane-nodrc` target drops DRC for iteration.

What scrolls by while it runs

- LibreLane prints a banner and numbered steps (`Step 1: Yosys.Synthesis`, ...).
- Yosys: inferred cell counts + area.
- OpenROAD: floorplan, PDN, placement, CTS, routing, STA.
- Magic + KLayout: DRC (the slow parts).
- Netgen: LVS.
- Final banner: run directory + pointer to `metrics.csv`.

Screenshot placeholder

`slides/figures/librelane_run_output.png`

Tail of a successful `make librelane` run.

What scrolls by while it runs
LibreLane prints a banner and numbered steps (Step 1: Yosys.Synthesis, ...).
Yosys: inferred cell counts + area.
OpenROAD: floorplan, PDN, placement, CTS, routing, STA.
Magic + KLayout: DRC (the slow parts).
Netgen: LVS.
Final banner: run directory + pointer to metrics.csv.
Screenshot placeholder
slides/figures/librelane_run_output.png
Tail of a successful make librelane run.

The log is dense but readable. Every step prints a header like `== Step 42: OpenROAD.Floorplan` so you can grep. Warnings during routing repair iterations are expected; failures manifest as the flow stopping abruptly on a step boundary.

The final banner tells you two things: the timestamped run directory (where the full logs and intermediate DEFs live) and the path to `metrics.csv`. Bookmark both when presenting live.

Where artifacts land

```
template/
  librelane/runs/RUN_<timestamp>/   # per-run: logs, DEFs, reports
  final/                            # merged: GDS, LEF, SDC, SDF, SPEF,
                                    # metrics.csv, render/chip_top.png
```

Useful after a successful run:

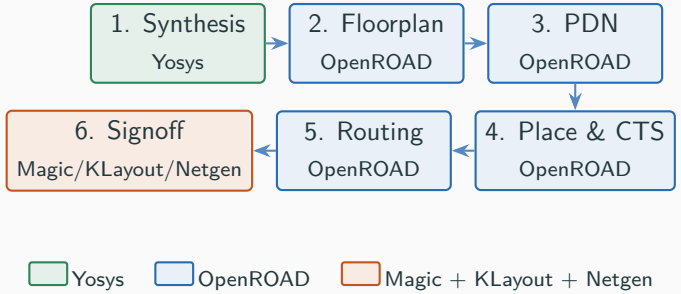
- `final/gds/chip_top.gds` – the deliverable.
- `final/metrics.csv` – one row per metric.
- `final/render/chip_top.png` – auto-render for posters.

Where artifacts land
template/
librelane/runs/RUN_<timestamp>/ # per-run: logs, DEFs, reports
final/ # merged: GDS, LEF, SDC, SDF, SPEF,
metrics.csv, render/chip_top.png
Useful after a successful run:
final/gds/chip_top.gds – the deliverable.
final/metrics.csv – one row per metric.
final/render/chip_top.png – auto-render for posters.

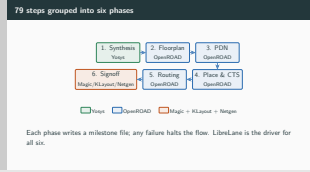
The split matters: `runs/` is every flow artifact organised per run (useful for bisecting), `final/` is the curated deliverable set that LibreLane copies via its `-save-views-to` flag. You almost never dig into `runs/` unless something failed.

Practical tip: `runs/` balloons fast. Add it to `.gitignore`. Commit only a trimmed `final/metrics.csv` as a regression baseline. Add a CI job that greps the next run's `metrics.csv` for the key fields and compares – that is how you catch silent regressions.

79 steps grouped into six phases



Each phase writes a milestone file; any failure halts the flow. LibreLane is the driver for all six.



Recap diagram before the deep-dive section. Audience already saw the pipeline in section 1 – the difference now is (a) the colour key makes "which tool owns which phase" visible at a glance, and (b) they have seen the command that starts it.

Point at each box and promise: "the next six slides tell you what actually happens inside these, and which underlying tool LibreLane is calling."

Why 79 steps and not 6? LibreLane breaks the phases into many auxiliary passes (netlist cleanup, antenna repair, resizing, multiple DRC runs). The 6-phase model is the mental pipeline; the 79 steps are the implementation. Each of the 79 maps to exactly one tool.

Flow stages up close

Flow stages up close

Phase 1 – Synthesis (Yosys)

LibreLane driving: **Yosys**

- RTL → gate-level netlist using the GF180 standard-cell library (`gf180mcu_fd_sc_mcu7t5v0`).
- Emits instance counts, estimated area, inferred FF / LUT shapes.
- Produces the first mapped `chip_top.v` that feeds every downstream step.

Reference run numbers:

```
design__instance__count,251615
design__instance__count__stdcell,84417
```


Phase 1 – Synthesis (Yosys)
LibreLane driving: Yosys
RTL → gate-level netlist using the GF180 standard-cell library (gf180mcu_fd_sc_mcu7t5v0).
Emits instance counts, estimated area, inferred FF / LUT shapes.
Produces the first mapped chip_top.v that feeds every downstream step.
Reference run numbers:
design__instance__count,251615
design__instance__count__stdcell,84417

Yosys is fast (seconds) and deterministic. The 251 k instance count looks huge – it is dominated by ∼1100 padring cells and 81k tap cells inserted for well-tap density. The meaningful number is `instance_count__stdcell` (84k).

If synthesis fails, it is almost always a Verilog issue (unresolved module, missing port, wrong bit width). LibreLane surfaces Yosys errors verbatim; grep the log for `ERROR:` to find the offending line.

Phase 2 – Floorplan & padring

LibreLane driving: **OpenROAD**

- `OpenROAD.Floorplan` reads `DIE_AREA` / `CORE_AREA` and creates an empty chip frame.
- `OpenROAD.Padring` places I/O pads per `PAD_SOUTH/...` order.
- `ODB.ManualMacroPlacement` drops the SRAMs at the coordinates from `config.yaml`.
- `CutRows` + `TapcellInsertion` prepare standard-cell rows.

Screenshot placeholder

`slides/figures/floorplan.png`

OpenROAD: post-floorplan view (padring + SRAM placeholders).

Phase 2 – Floorplan & padring
LibreLane driving: OpenROAD
OpenROAD.Floorplan reads DIE_AREA / CORE_AREA and creates an empty chip frame.
OpenROAD.Padring places I/O pads per PAD_SOUTH/... order.
ODB.ManualMacroPlacement drops the SRAMs at the coordinates from config.yaml.
CutRows + TapcellInsertion prepare standard-cell rows.
Screenshot placeholder
slides/figures/floorplan.png
OpenROAD: post-floorplan view (padring + SRAM placeholders).

This is the phase that turns YAML into geometry. If the pad list is wrong, the padring does not close and floorplan fails loudly. If `CORE_AREA` is too tight, `CutRows` creates too few rows and placement fails later. If an SRAM instance coordinate overlaps the padring, `ManualMacroPlacement` fails here.

Tap cells (well ties) are required by the PDK at a fixed pitch. Forget them and downstream LVS complains. LibreLane inserts them automatically.

Phase 3 – Power Distribution Network

LibreLane driving: **OpenROAD**

- OpenROAD runs `pdn_cfg.tcl` to draw the grid.
- **Followpin** wires: thin VDD/VSS along every std-cell row.
- **Straps**: Metal2 vertical + Metal3 horizontal across the core.
- **Core ring**: loop around the core that connects to pads.
- Per-macro grids stitch SRAM power pins to the main grid.
- Checker target: `design__power_grid_violation__count = 0`.

Screenshot placeholder

`slides/figures/pdn.png`

Core PDN with straps and ring visible.

Phase 3 – Power Distribution Network
LibreLane driving: OpenROAD
OpenROAD runs pdn_cfg.tcl to draw the grid.
Followpin wires: thin VDD/VSS along every std-cell row.
Straps: Metal2 vertical + Metal3 horizontal across the core.
Core ring: loop around the core that connects to pads.
Per-macro grids stitch SRAM power pins to the main grid.
Checker target: design__power_grid_violation__count = 0.
Screenshot placeholder
slides/figures/pdn.png
Core PDN with straps and ring visible.

Power is the one pass where you *want* to touch Tcl. `pdn_cfg.tcl` declares voltage domains and per-macro power grids; OpenROAD's PDN generator consumes it verbatim. Voltage domain naming must match `VDD_NETS`/`GND_NETS` in `config.yaml` or the tool refuses to start the phase. Signoff will re-check the grid (PDNSim + IR drop). A grid with zero *construction* violations can still fail IR; if that happens, either widen straps or tighten pitch.

Phase 4 – Placement & CTS

LibreLane driving: **OpenROAD**

- **Global placement**: approximate positions (RePlAce).
- **Detailed placement**: legalise onto std-cell rows.
- **Clock Tree Synthesis**: buffer tree from `clk_pad/Y` to every flop, minimising skew.
- **Post-CTS resizing**: fixes hold violations introduced by CTS.

Reference run:

```
design__instance__count__class:clock_buffer,30
clock__skew__worst_setup,0.319   # ns
clock__skew__worst_hold, -0.328  # ns
```


Phase 4 – Placement & CTS
LibreLane driving: OpenROAD
Global placement: approximate positions (RePlAce).
Detailed placement: legalise onto std-cell rows.
Clock Tree Synthesis: buffer tree from clk_pad/Y to every flop, minimising skew.
Post-CTS resizing: fixes hold violations introduced by CTS.
Reference run:
design__instance__count__class:clock_buffer,30
clock__skew__worst_setup,0.319 # ns
clock__skew__worst_hold, -0.328 # ns

Placement is iterative. Global placement runs on a quadratic wirelength model; detailed placement snaps to legal rows. CTS is where a well-behaved flow earns its keep: 30 clock buffers, skew below half a nanosecond, no PLL needed for 25 MHz.

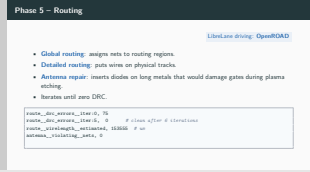
Common failure: "too many flops, not enough rows" – placement congestion exceeds a threshold. Fix by enlarging `CORE_AREA` or adjusting `PL_TARGET_DENSITY_PCT` (lower = more spread, less congestion, bigger die).

Phase 5 – Routing

LibreLane driving: **OpenROAD**

- **Global routing**: assigns nets to routing regions.
- **Detailed routing**: puts wires on physical tracks.
- **Antenna repair**: inserts diodes on long metals that would damage gates during plasma etching.
- Iterates until zero DRC.

```
route__drc_errors__iter:0, 75
route__drc_errors__iter:5,  0      # clean after 6 iterations
route__wirelength__estimated, 153555  # um
antenna__violating__nets, 0
```

Routing is where designs blow up their wall-time budgets. OpenROAD's TritonRoute iterates: global route, detailed route, DRC, repair, repeat. Our run converges in 6 iterations; busy designs can take 20+ on a congested die.

Antennas are a fabrication concern, not a timing one: long metal wires accumulate plasma charge during etching, which can punch through a gate oxide. Diodes bleed the charge. LibreLane's repair pass inserts them automatically.

If routing fails to converge, the fix is almost always the floorplan (more core area, lower density) or the congestion-vs-density knob.

Phase 6 – Signoff (geometry + electrical)

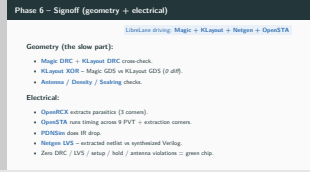
LibreLane driving: **Magic + KLayout + Netgen + OpenSTA**

Geometry (the slow part):

- **Magic DRC** + **KLayout DRC** cross-check.
- **KLayout XOR** – Magic GDS vs KLayout GDS (*0 diff*).
- **Antenna / Density / Sealring** checks.

Electrical:

- **OpenRCX** extracts parasitics (3 corners).
- **OpenSTA** runs timing across 9 PVT + extraction corners.
- **PDNSim** does IR drop.
- **Netgen LVS** – extracted netlist vs synthesized Verilog.
- Zero DRC / LVS / setup / hold / antenna violations = green chip.



Signoff is the "is this really manufacturable?" gate. Two independent DRC engines (Magic and KLayout) cross-check each other; an XOR confirms their geometries match. Any mismatch is a show-stopper.

Electrical: RCX extracts RC parasitics from the final GDS. STA then re-checks timing with those real parasitics at every corner. IR drop checks the power grid under dynamic load. LVS is non-negotiable – it is the single check that proves the layout implements the circuit you synthesized.

Every check here writes one metric into `metrics.csv`. The next section is about reading that file.

Results, pitfalls, and next steps

Results, pitfalls, and next steps

Headline numbers – reference run

Metric	Value
Die area	3.93×5.12 mm
Core area	3.05×4.24 mm
Core utilisation	10.45 %
Standard-cell instances	84 417
Total instances (incl. padring, taps)	251 615
Macros	4 (2× SRAM + ID + logo)
I/O pads	58
Routed wirelength	153.5 mm
Total power (tt_025C_5v00)	34.3 mW
Clock period / frequency	40 ns / 25 MHz

Source: `final/metrics.csv`, run 2026-04-13.

Headline numbers – reference run

Metric	Value
Die area	3.93×5.12 mm
Core area	3.05×4.24 mm
Core utilisation	10.45 %
Standard-cell instances	84 417
Total instances (incl. padring, taps)	251 615
Macros	4 (2× SRAM + ID + logo)
I/O pads	58
Routed wirelength	153.5 mm
Total power (tt_025C_5v00)	34.3 mW
Clock period / frequency	40 ns / 25 MHz

Source: `final/metrics.csv`, run 2026-04-13.

These numbers anchor every conversation. Low core utilisation (10%) is normal for a full-chip padring design: the die is sized for the shuttle slot, not the logic. If the goal were area, you would shrink `DIE_AREA` and drop the padring.

Power at 34 mW at 25 MHz, 5 V: typical for 180 nm with this much logic. Scale linearly with frequency for first-order estimates. Mention these numbers so the audience has a feel for "what good looks like" before they run their own design.

Signoff checklist – the things that must be zero

Check	Violations
Magic DRC	0
KLayout DRC	0
KLayout XOR (Magic vs KLayout)	0
KLayout antenna / density	0
Magic illegal overlap	0
Netgen LVS	clean
Setup / Hold (all 9 STA corners)	0 / 0
Routing DRC (final iteration)	0
IR drop	0
Max slew (ss_125C corner only)	3 (warning)
Max cap (all corners)	100–103 (warning)

Chip is manufacturable; the warnings are slow-corner fine-tuning.

Signoff checklist – the things that must be zero
Check Violations
Magic DRC 0
KLayout DRC 0
KLayout XOR (Magic vs KLayout) 0
KLayout antenna / density 0
Magic illegal overlap 0
Netgen LVS clean
Setup / Hold (all 9 STA corners) 0 / 0
Routing DRC (final iteration) 0
IR drop 0
Max slew (ss_125C corner only) 3 (warning)
Max cap (all corners) 100–103 (warning)
Chip is manufacturable; the warnings are slow-corner fine-tuning.

Signoff is a binary gate. Everything above the line must be zero. Warnings below the line are shipping-acceptable for a first silicon spin but worth investigating before re-tapeout. The distinction between "violation" and "warning" is tool-specific – LibreLane normalises them into `metrics.csv` so you can script gates in CI.

If you see non-zero LVS, stop. LVS failure means the GDS does not match the netlist – the chip will not behave like your simulation, regardless of DRC or timing.

Reading `metrics.csv` in one grep

```
grep -E '^(magic__drc|klayout__drc|lvs|timing__(setup|hold)_vio__count$|design__(violations|
    die__area|instance__count)|power__total)' \
    ~/eda/designs/template/final/metrics.csv
```

`metrics.json` has the same data in a friendlier shape for scripts. Commit a trimmed copy as a regression baseline.

This is the presenter's cheat-sheet slide. Keep it on screen during Q&A. The recipe grabs: DRC counts from both Magic and KLayout, LVS status, setup/hold violation counts, die area, instance count, total power. Enough to know a flow is green without opening a GUI.

CI pattern: on every push, run LibreLane, grep these fields, compare to baseline with a tolerance band on numeric fields and an exact match on violation counts. Fails loud the moment someone breaks the flow.

Four pitfalls you *will* hit

1. Wrong PDK active. Error mentions `sg13g2_stdcell`. **Fix:** `source sak-pdk-script.sh gf180mcuD gf180mcu_fd_sc_mcu7t5v0` before `make`.

2. Missing SRAM / I/O files. Error says path `'/foss/pdks/ihp-sg13g2/libs.ref/gf180mcu_fd_ip_sram/...'` does not exist. **Fix:** pass `PDK_ROOT=/foss/designs/template/gf180mcu` explicitly.

3. Unknown I/O cell `gf180mcu_ws_io__...`. **Fix:** clone the wafer-space PDK fork (`make clone-pdk`).

4. Flow is absurdly slow. Magic DRC ∼32 min + KLayout antenna ∼7 min dominate runtime. **Fix for iteration:** `make librelane-nodrc`; never hand in a chip without one full DRC run.

Four pitfalls you *will* hit
1. Wrong PDK active. Error mentions sg13g2_stdcell. Fix: source sak-pdk-script.sh gf180mcuD gf180mcu_fd_sc_mcu7t5v0 before make.
2. Missing SRAM / I/O files. Error says path '/foss/pdks/ihp-sg13g2/libs.ref/gf180mcu_fd_ip_sram/...' does not exist. Fix: pass PDK_ROOT=/foss/designs/template/gf180mcu explicitly.
3. Unknown I/O cell gf180mcu_ws_io__... Fix: clone the wafer-space PDK fork (make clone-pdk).
4. Flow is absurdly slow. Magic DRC ∼32 min + KLayout antenna ∼7 min dominate runtime. Fix for iteration: make librelane-nodrc; never hand in a chip without one full DRC run.

These four cover >80 % of the first-timer failure modes.

Pitfalls 1 and 2 look like "the PDK is wrong" but have different fixes: 1 is *shell env*, 2 is *Make variable*. Teach people to check both.

Pitfall 3 is the reason the wafer-space PDK fork exists at all – upstream open_pdks GF180 does not ship the custom padring cells.

Pitfall 4 is a philosophy point: iteration speed matters. Use `nodrc` for design iteration, then run the full flow before tapeout. This mirrors how unit tests vs integration tests work in software.

What to try next

1. **Swap** `chip_core.sv` for your own logic (same ports). Re-run. You now have a custom chip.
2. **Change** `CLOCK_PERIOD` and watch setup/hold slack move.
3. **Swap the slot** to `1x0p5` or `0p5x0p5`, adjust pad lists, re-run – see the floorplan scale.
4. **Submit to a shuttle**: Tiny Tapeout, wafer-space MPW. The GDS you produced is fab-ready.

Further reading: LibreLane docs `https://librelane.readthedocs.io/en/stable/` · Image `https://github.com/iic-jku/IIC-OSIC-TOOLS` · Template `https://github.com/wafer-space/gf180mcu-project-template` · Wafer-space PDK fork `https://github.com/wafer-space/gf180mcu` · GF180 PDK `https://github.com/google/gf180mcu-pdk`.

What to try next

1. Swap chip_core.sv for your own logic (same ports). Re-run. You now have a custom chip.
2. Change CLOCK_PERIOD and watch setup/hold slack move.
3. Swap the slot to 1x0p5 or 0p5x0p5, adjust pad lists, re-run – see the floorplan scale.
4. Submit to a shuttle: Tiny Tapeout, wafer-space MPW. The GDS you produced is fab-ready.

Further reading: LibreLane docs https://librelane.readthedocs.io/en/stable/ · Image https://github.com/iic-jku/IIC-OSIC-TOOLS · Template https://github.com/wafer-space/gf180mcu-project-template · Wafer-space PDK fork https://github.com/wafer-space/gf180mcu · GF180 PDK https://github.com/google/gf180mcu-pdk.

Call-to-action slide. The cheapest way to learn is to take the template, replace `chip_core.sv` with "hello world" logic (blinky, counter, UART echo), and re-run. 45 minutes later you have a real GDS.

For presenter: point people to the companion Jupyter notebook (`demo/rtl2gds_counter.ipynb`) which walks through a minimal counter in 7 steps – a faster iteration target than the full-chip template. It is fully self-contained; drop it anywhere and run.